KOTLIN MULTIPLATFORM · ANDROID · CLEAN ARCHITECTURE

Market KMP

Guia Completo do Projeto

Do conceito à implementação — tudo que foi construído e por quê

PROJETO	PLATAFORMA	LINGUAGEM	ANO
Lista de Compras	Android · iOS ready	Kotlin 2.0	2026

Sumário

Este guia cobre toda a construção do app Market — do zero ao produto final.

01 O Problema e a Solução

02 O que é Kotlin Multiplatform (KMP)?

Como o KMP funciona por dentro · expect/actual · Targets iOS

03 Arquitetura do Projeto — Clean Architecture

Camadas · Fluxo de dados · Por que separar?

04 Camada de Domínio — O Coração Compartilhado

GroceryItem · Category · Repository Interface · Use Cases

05 Camada de Dados — Room e Repository

Entity · DAO · Banco de dados · Mapeamento

06 Injeção de Dependências com Koin

07 Camada de Apresentação — ViewModel e State

StateFlow · combine() · UiState · Ações

08 Interface — Jetpack Compose

Tela principal · Componentes · Animações · Material 3

09 Funcionalidades do App

10 iOS — O que está pronto e o que falta

11 Stack Tecnológica e Próximos Passos

01

CONTEXTO

O Problema e a Solução

Todo casal que divide as compras do mercado já passou por isso: a lista está no celular de quem ficou em casa, e quem está no mercado não sabe o que pegar — ou compra coisa duplicada.

A ideia do app

O Market resolve isso de forma simples: **uma pessoa cadastra os itens** (em casa, com calma), e **a outra marca na hora da compra**. Sem ligações, sem mensagens de WhatsApp com listas bagunçadas.

🎯 **Decisão de produto**

O app foi pensado para dois usuários com papéis distintos: quem planeja e quem executa. Essa divisão de papéis influenciou diretamente o design da interface — a tela principal é otimizada para marcar itens com um toque rápido.

Por que construir com KMP?

O objetivo final é ter o mesmo app rodando no Android de uma pessoa e no iPhone da outra. Escrever a lógica duas vezes — uma em Kotlin, outra em Swift — seria ineficiente e arriscado (risco de comportamentos diferentes).

Com KMP, a lógica de negócio é escrita uma vez em Kotlin e funciona em ambas as plataformas. A UI permanece nativa em cada plataforma (Compose no Android, SwiftUI no iOS).

📚 **Princípio: Write once, run anywhere (a lógica)**

"Don't Repeat Yourself" (DRY) aplicado ao desenvolvimento mobile multiplataforma. O que muda por plataforma é apenas como o usuário interage — a regra do que acontece permanece a mesma.

O que foi construído

Um app Android completo e funcional, com toda a camada de negócio estruturada para ser reutilizada no iOS:

✓ **App Android completo**

Tela de lista com todas as funcionalidades: CRUD, filtros, busca, progresso.

✓ **Domínio compartilhado**

Modelos, use cases e interface de repositório prontos para iOS.

Framework iOS gerado

O shared.xcframework pode ser importado em qualquer projeto Xcode.

Arquitetura escalável

Clean Architecture com separação clara de responsabilidades.

02

TEORIA + PRÁTICA

O que é Kotlin Multiplatform?

KMP é a tecnologia da JetBrains que permite compartilhar código Kotlin entre diferentes plataformas — Android, iOS, Desktop, Web — sem abrir mão de UIs nativas em cada uma.

📚 **Como funciona por dentro**

O compilador Kotlin tem backends diferentes para cada plataforma. Para Android, gera bytecode JVM normal. Para iOS, usa LLVM para compilar Kotlin diretamente para código nativo ARM — sem JVM no iPhone. Para Web, transpila para JavaScript.

Estrutura de Source Sets

O módulo `shared` é dividido em "source sets" — cada um compila para uma plataforma específica. O código em `commonMain` é compilado para todas.

```
shared/src/
├── commonMain/   ← compilado para TODAS as plataformas
│   └── domain/   ← modelos, use cases, interface repositório
│
├── androidMain/  ← código exclusivo Android (herda commonMain)
│   └── Platform.android.kt
│
└── iosMain/      ← código exclusivo iOS (herda commonMain)
    └── Platform.ios.kt
```

O padrão expect / actual

Quando precisamos de comportamento diferente por plataforma, usamos `expect` no commonMain para declarar o contrato, e `actual` em cada plataforma para implementar.

```kotlin
// commonMain/Platform.kt — declara o contrato
expect fun getPlatformName(): String

// androidMain/Platform.android.kt — implementação Android
actual fun getPlatformName(): String = "Android"

// iosMain/Platform.ios.kt — implementação iOS
actual fun getPlatformName(): String = "iOS"
```

🛠️ **No projeto Market**

O expect/actual é usado minimamente — apenas para detecção de plataforma. O mais importante é que **Flow e Coroutines estão em commonMain**, o que significa que toda a reatividade do app (dados em tempo real do banco) funciona multiplataforma sem nenhuma adaptação.

Configuração iOS no build.gradle.kts

```
kotlin {
    // 3 targets iOS para cobrir todos os casos:
    iosX64()            // Simulador em Mac Intel
    iosArm64()          // iPhone físico
    iosSimulatorArm64() // Simulador em Mac M1/M2/M3

    targets.withType<KotlinNativeTarget>() {
        binaries.framework {
            baseName = "shared"
            isStatic = true // framework estático → mais simples de integrar
        }
    }
}
```

💡 **Por que isStatic = true?**

Um framework estático é embutido no binário do app iOS em tempo de compilação. Mais simples de distribuir e sem overhead de carregamento em tempo de execução. A alternativa (dinâmico) é melhor quando múltiplos apps compartilham o mesmo framework — não é o caso aqui.

03

ARQUITETURA

Clean Architecture

Clean Architecture organiza o código em camadas concêntricas onde a regra de negócio não depende de nada externo — banco de dados, UI, framework. Isso torna o código testável, substituível e duradouro.

As três camadas do projeto

🖥️

Presentation — androidApp/presentation/

Exibe dados para o usuário e captura suas ações. Só sabe sobre o domínio.

`GroceryListScreen.kt` · `GroceryViewModel.kt` · `GroceryItemCard.kt` · `AddItemBottomSheet.kt`

↕

🧠

Domain — shared/commonMain/domain/

Regras de negócio puras. Não conhece Android, Room ou Compose. É o núcleo compartilhado.

`GroceryItem.kt` · `Category.kt` · `GroceryRepository.kt` · `GroceryUseCases.kt`

↕

🗄️

Data — androidApp/data/

Implementa como os dados são armazenados e recuperados. Conhece Room, mas implementa a interface do domínio.

`GroceryDatabase.kt` · `GroceryItemDao.kt` · `GroceryItemEntity.kt` · `GroceryRepositoryImpl.kt`

A Regra de Dependência

As dependências apontam sempre para **dentro** — em direção ao domínio. O domínio nunca importa nada das camadas externas.

⚠️

O que isso significa na prática

Se amanhã quisermos trocar Room por SQLDelight (para compartilhar o banco com iOS), só precisamos criar uma nova implementação de `GroceryRepository`. Os use cases, o ViewModel e a UI **não mudam nada** — eles não sabem que o banco mudou.

Fluxo completo de uma ação

Exemplo: usuário toca no checkbox de um item para marcá-lo como coletado.

1. **UI dispara evento**

   `GroceryItemCard` chama `onToggle(item)` quando o checkbox é tocado.

2. **ViewModel processa**

   `toggleItem(item)` inverte o campo `isChecked` e chama `useCases.upsertItem()`.

3. **Use Case executa**

   `UpsertGroceryItemUseCase` delega para `repository.upsertItem(item)`.

4. **Repositório persiste**

   `GroceryRepositoryImpl` converte o item para Entity e chama `dao.upsertItem(entity)`.

5. **Room notifica via Flow**

   O banco emite nova lista pelo `Flow` do DAO. O `combine()` no ViewModel recalcula o `UiState`. O Compose recompõe automaticamente.

04

SHARED/COMMONMAIN

Camada de Domínio

O domínio é o módulo mais valioso do projeto — é onde vive a lógica que não muda independente de plataforma, framework ou banco de dados.

GroceryItem — o modelo central

Uma `data class` em Kotlin é perfeita para modelos de domínio: imutável por padrão, com `equals()`, `hashCode()` e `copy()` gerados automaticamente.

```
data class GroceryItem(
    val id: String,          // UUID — gerado no ViewModel na criação
    val name: String,        // "Leite integral"
    val quantity: Double,    // 1.5 → "1.5 L"
    val unit: String,        // "L", "kg", "un", "pct"...
    val category: Category, // enum com emoji e label PT-BR
    val isChecked: Boolean, // false = no carrinho / true = coletado
    val note: String,        // "Marca Italac" — opcional
    val createdAt: Long      // System.currentTimeMillis()
)
```

💡 **Por que Double para quantidade?**

Para suportar valores como 0.5 kg ou 1.5 L. O app permite incrementos de 0.5 quando a quantidade é menor que 1, e incrementos de 1 acima disso. Isso é tratado na UI — o domínio apenas armazena o valor.

Category — enum rico

Em vez de um enum simples, cada categoria carrega seu próprio **label em português** e **emoji** — dados de apresentação colocados no domínio porque são intrínsecos à categoria, não detalhes de UI.

```
enum class Category(val label: String, val emoji: String) {
    PRODUCE("Hortifruti", "🥦"),
    DAIRY("Laticínios", "🥛"),
    MEAT("Carnes", "🥩"),
    BAKERY("Padaria", "🍞"),
    BEVERAGES("Bebidas", "🧃"),
    FROZEN("Congelados", "🧊"),
    PERSONAL_CARE("Higiene", "🧴"),
    CLEANING("Limpeza", "🧹"),
    SNACKS("Snacks", "🍿"),
```

```
    OTHER("Outros", "🛒")
}
```

Repository Interface — o contrato

A interface define *o que* o repositório faz, sem dizer *como*. O domínio depende desta interface — nunca da implementação concreta (Room).

```
interface GroceryRepository {
    fun getAllItems(): Flow<List<GroceryItem>>  // reativo — emite quando o banco muda
    suspend fun getItemById(id: String): GroceryItem?
    suspend fun upsertItem(item: GroceryItem)     // insert ou update
    suspend fun deleteItem(id: String)
    suspend fun clearCheckedItems()               // deleta todos os marcados
}
```

Use Cases — uma responsabilidade cada

Cada use case encapsula **uma única operação de negócio**. O padrão `operator fun invoke()` permite chamá-los como funções diretas.

```
// Exemplo — GetGroceryItemsUseCase
class GetGroceryItemsUseCase(private val repository: GroceryRepository) {
    operator fun invoke(): Flow<List<GroceryItem>> = repository.getAllItems()
}

// No ViewModel — chamada limpa, parece uma função normal:
useCases.getItems()  // em vez de useCases.getItems.invoke()
```

🛠️ **Agrupamento em data class**

Todos os 4 use cases são agrupados em `GroceryUseCases(getItems, upsertItem, deleteItem, clearChecked)`. Assim o ViewModel recebe uma única dependência em vez de quatro — e o Koin injeta tudo de uma vez.

05

ANDROIDAPP/DATA

Camada de Dados — Room

A camada de dados é a implementação concreta do repositório. Ela conhece o Room, converte entre Entity e domínio, e expõe os dados como Flow reativo.

Por que precisamos de uma Entity separada?

O domínio tem `GroceryItem` com `category: Category` (enum). O Room não sabe armazenar enums diretamente — armazena Strings. A Entity é a representação "achatada" do domínio para o banco.

GroceryItem (domínio)	GroceryItemEntity (banco)
`category: Category`	`category: String` ("DAIRY")
Objeto rico com métodos	Data class simples, anotada com @Entity
Imutável, Kotlin puro	Conhece Room, Android-only

O DAO — queries que retornam Flow

A query principal tem uma ordenação inteligente que garante a experiência certa: itens não marcados primeiro, agrupados por categoria, em ordem alfabética.

```
@Dao
interface GroceryItemDao {

    @Query("""
        SELECT * FROM grocery_items
        ORDER BY isChecked ASC,   -- não marcados primeiro
                 category ASC,    -- agrupa por categoria
                 name ASC         -- alfabético dentro da categoria
    """)
    fun getAllItems(): Flow<List<GroceryItemEntity>>  // reativo!

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun upsertItem(item: GroceryItemEntity)  // insert ou update

    @Query("DELETE FROM grocery_items WHERE id = :id")
    suspend fun deleteItem(id: String)

    @Query("DELETE FROM grocery_items WHERE isChecked = 1")
    suspend fun clearCheckedItems()  // limpa todos marcados de uma vez
}
```

📚 **Como o Flow do Room funciona**

Quando Room retorna um `Flow`, ele automaticamente emite uma nova lista toda vez que qualquer linha da tabela muda. Não é necessário chamar "refresh" manualmente — a UI se atualiza sozinha quando o banco muda.

GroceryRepositoryImpl — mapeamento bidirecional

```
class GroceryRepositoryImpl(database: GroceryDatabase) : GroceryRepository {

    private val dao = database.groceryItemDao()

    // Entity → Domínio (com enum.valueOf para recriar o enum)
    private fun GroceryItemEntity.toDomain() = GroceryItem(
        id, name, quantity, unit,
        category = Category.valueOf(category),  // "DAIRY" → Category.DAIRY
        isChecked, note, createdAt
    )

    // Domínio → Entity (enum para String)
    private fun GroceryItem.toEntity() = GroceryItemEntity(
        id, name, quantity, unit,
        category = category.name,  // Category.DAIRY → "DAIRY"
        isChecked, note, createdAt
    )

    override fun getAllItems() = dao.getAllItems().map { list ->
        list.map { it.toDomain() }  // mapeia cada entity para domínio
    }
}
```

06

ANDROIDAPP/DI

Injeção de Dependências com Koin

Injeção de dependências resolve um problema simples: como fornecer os objetos certos para quem precisa deles, sem que cada classe precise saber como construir suas próprias dependências.

📚 **O problema sem DI**

Sem injeção, o ViewModel precisaria criar o banco, criar o repositório, criar os use cases — dentro do seu construtor. Isso acopla tudo, impossibilita testes e complica mudanças. Com DI, o ViewModel *pede* o que precisa, e o container entrega.

O módulo Koin do projeto

```
val appModule = module {

    // Banco — singleton (uma instância para todo o app)
    single { GroceryDatabase.create(androidContext()) }

    // Repositório — singleton, implementa a interface do domínio
    single<GroceryRepository> { GroceryRepositoryImpl(get()) }

    // Use Cases — singleton, get() busca GroceryRepository automaticamente
    single {
        GroceryUseCases(
            getItems    = GetGroceryItemsUseCase(get()),
            upsertItem  = UpsertGroceryItemUseCase(get()),
            deleteItem  = DeleteGroceryItemUseCase(get()),
            clearChecked = ClearCheckedItemsUseCase(get())
        )
    }

    // ViewModel — factory (nova instância por tela)
    viewModel { GroceryViewModel(get()) }
}
```

single vs viewModel

Escopo	Comportamento	Quando usar
`single`	Uma instância compartilhada no app inteiro	Banco, repositório, use cases

Escopo	Comportamento	Quando usar
`viewModel`	Nova instância por ViewModel, destruída com ele	ViewModels — ligados ao ciclo de vida
`factory`	Nova instância a cada chamada get()	Objetos leves sem estado

Como o Koin é iniciado

```
// MarketApplication.kt
class MarketApplication : Application() {
    override fun onCreate() {
        super.onCreate()
        startKoin {
            androidLogger()
            androidContext(this@MarketApplication)
            modules(appModule)
        }
    }
}
```

Como a tela usa o ViewModel

```
// GroceryListScreen.kt — Koin injeta automaticamente
@Composable
fun GroceryListScreen(
    viewModel: GroceryViewModel = koinViewModel()  // ← magia do Koin + Compose
) { ... }
```

🛠️ **koinViewModel() vs viewModel()**

`koinViewModel()` é o equivalente do `viewModel()` padrão do Compose, mas usa o Koin para criar a instância. Isso significa que o ViewModel recebe suas dependências automaticamente — sem precisar de ViewModelFactory manual.

07

PRESENTATION/VIEWMODEL

ViewModel e Gerenciamento de Estado

O ViewModel é a ponte entre o domínio e a UI. Ele mantém o estado da tela, processa ações do usuário e expõe dados como StateFlow — que o Compose observa automaticamente.

O UiState — estado completo da tela

Todo o estado necessário para renderizar a tela fica em uma única data class imutável. Propriedades computadas (calculadas a partir dos dados) ficam como `val` dentro da data class.

```
data class GroceryUiState(
    val allItems: List<GroceryItem> = emptyList(),
    val isLoading: Boolean = true,
    val filter: FilterOption = FilterOption.ALL,
    val searchQuery: String = ""
) {
    // Computado: filtra por opção + texto de busca
    val filteredItems: List<GroceryItem> get() = allItems
        .filter { when (filter) {
            PENDING -> !it.isChecked; CHECKED -> it.isChecked; else -> true
        }}
        .filter { it.name.contains(searchQuery, ignoreCase = true) }

    // Computado: agrupa os filtrados por categoria
    val groupedItems: Map<Category, List<GroceryItem>> get() =
        filteredItems.groupBy { it.category }

    val checkedCount get() = allItems.count { it.isChecked }
    val totalCount   get() = allItems.size
    val progress     get() = if (totalCount == 0) 0f else checkedCount /
totalCount.toFloat()
}
```

combine() — o coração reativo do ViewModel

O estado final é uma **combinação reativa de 4 flows**. Qualquer mudança em qualquer um deles reconstrói o UiState automaticamente.

```
val uiState = combine(
    useCases.getItems(),  // Flow do banco — emite quando dados mudam
    _filter,              // StateFlow — filtro selecionado
    _searchQuery,         // StateFlow — texto da busca
    _isLoading            // StateFlow — estado de carregamento
) { items, filter, query, loading ->
```

```
    GroceryUiState(allItems = items, filter = filter,
                   searchQuery = query, isLoading = loading)
}.stateIn(
    scope = viewModelScope,
    started = SharingStarted.WhileSubscribed(5_000),  // ativo por 5s sem observadores
    initialValue = GroceryUiState()
)
```

💡 **Por que WhileSubscribed(5_000)?**

Quando o usuário gira a tela, o Composable é destruído e recriado. Sem o delay de 5 segundos, isso desconectaria e reconectaria o Flow do banco imediatamente. Com o delay, a conexão permanece por até 5 segundos — tempo suficiente para a rotação completar sem custo extra.

Como o ViewModel preserva dados na edição

```
fun updateItem(original: GroceryItem, name: String, ...) {
    viewModelScope.launch {
        useCases.upsertItem(
            original.copy(      // copy() preserva o que não mudou
                name = name.trim(),
                // id, createdAt e isChecked = preservados do original
            )
        )
    }
}
```

08

PRESENTATION/SCREEN + COMPONENTS

Interface — Jetpack Compose

Compose é um framework de UI declarativo: você descreve como a tela deve parecer dado um estado, e o Compose decide o que precisa ser atualizado. Não existe "atualizar o texto do botão" — você recompõe a tela com o novo estado.

GroceryListScreen — a tela principal

A tela observa o UiState e delega cada interação ao ViewModel. Estado local (modais abertos) fica no Composable com `remember`.

```
@Composable
fun GroceryListScreen(viewModel: GroceryViewModel = koinViewModel()) {
    val uiState by viewModel.uiState.collectAsStateWithLifecycle()

    // Estado local — só relevante para esta tela
    var showAddSheet         by remember { mutableStateOf(false) }
    var itemToEdit           by remember { mutableStateOf<GroceryItem?>(null) }
    var searchActive         by remember { mutableStateOf(false) }
    var showClearConfirmDialog by remember { mutableStateOf(false) }
    ...
}
```

📚 **collectAsStateWithLifecycle vs collectAsState**

`collectAsStateWithLifecycle()` é a versão lifecycle-aware: suspende a coleta quando o app vai para background (tela apagada, app minimizado). Isso economiza recursos — o Flow não processa dados que não serão exibidos.

GroceryItemCard — o componente mais rico

→ **SwipeToDismissBox**

Detecta swipe da direita para a esquerda com threshold de 40%. Exibe fundo vermelho com ícone de lixeira. Não deleta imediatamente — apenas abre o dialog de confirmação.

→ **AlertDialog de confirmação**

Pergunta "Deseja remover X da lista?" com botão vermelho "Remover" e "Cancelar". Só deleta após confirmação explícita — evita exclusões acidentais.

→ **Estilo condicional ao estado**

Quando `isChecked = true`: nome com strikethrough, opacidade reduzida, card com cor diferente. Feedback visual imediato sem navegação.

AddItemBottomSheet — formulário inteligente

O mesmo componente funciona para adicionar e editar, controlado pela prop `initialItem`:

Campo	Comportamento
Nome	Obrigatório, validação na tentativa de salvar, autoCapitalize
Quantidade	Botões +/– com incremento inteligente (0.5 abaixo de 1, 1 acima)
Unidade	10 chips em FlowRow — quebra de linha automática
Categoria	Grid 2 colunas com emoji + label, seleção visual com borda
Nota	Opcional, campo de texto simples

Animações e polish

animateItem() — lista · animateContentSize() · pulsação no EmptyState · cores por categoria · sticky headers · barra de progresso arredondada · celebração 🎉 a 100%

09

PRODUTO

Funcionalidades do App

10 funcionalidades construídas com foco na experiência real de uso — do cadastro em casa até a marcação no mercado.

1 Criar item

Bottom sheet com formulário completo. UUID gerado automaticamente. Timestamp capturado na criação.

2 Listar itens

LazyColumn com sticky headers por categoria. Ordenação: não marcados primeiro, por categoria, por nome.

3 Editar item

Ícone de lápis abre o mesmo bottom sheet pré-preenchido. Preserva ID, data de criação e status de marcação.

4 Excluir item

Swipe para a esquerda revela fundo vermelho. Dialog de confirmação previne exclusões acidentais.

5 Marcar/desmarcar

Checkbox com toque único. Strikethrough no nome + opacidade reduzida + item vai para o final da lista.

6 Limpar marcados

Botão DeleteSweep aparece quando há itens marcados. Dialog mostra contagem exata antes de confirmar.

7 Filtrar

Três chips: Todos / Pendentes / Feitos. Estado mantido no ViewModel — sobrevive rotação de tela.

8 Buscar

DockedSearchBar com filtro em tempo real. Case-insensitive. Combinado com o filtro ativo.

9 Progresso

Card com "X de Y itens", porcentagem e barra linear animada. "Tudo coletado! 🎉" quando 100%.

10 Estado vazio

Ícone pulsante, mensagem contextual (lista vazia vs. nenhum resultado para a busca).

🛠️ **Decisão de UX: confirmação em ações destrutivas**

Tanto a exclusão individual (swipe) quanto a exclusão em lote (limpar marcados) têm AlertDialog de confirmação. A diferença é que o "limpar marcados" mostra a contagem exata de itens que serão removidos — contextualiza o impacto antes da ação.

10

KMP · IOS

iOS — O que está pronto e o que falta

O app Android está 100% funcional. A parte iOS está na metade mais fácil do caminho — o núcleo (lógica) está pronto, falta a casca (UI).

O que o iOS já herda hoje

O que está pronto	Arquivo
Modelo GroceryItem com todos os campos	shared/domain/model/GroceryItem.kt
Enum Category com labels PT-BR e emojis	shared/domain/model/Category.kt
Interface GroceryRepository	shared/domain/repository/
Todos os 4 use cases	shared/domain/usecase/
Flow reativo (kotlinx-coroutines-core)	build.gradle.kts commonMain
Framework estático gerado (3 targets)	binaries.framework { isStatic=true }

O que precisa ser construído para iOS

1. **Criar projeto SwiftUI no Xcode**
   Projeto iOS novo, importar o `shared.xcframework` como dependência local.

2. **Implementar GroceryRepository em Swift**
   Criar `GroceryRepositoryImpl.swift` usando Core Data ou SwiftData. Mesma interface, implementação nativa iOS.

3. **Adaptar Flow para Swift com SKIE ou KMP-NativeCoroutines**
   `Flow<List<GroceryItem>>` precisa ser exposto como `AsyncStream` no Swift. Bibliotecas como SKIE fazem isso automaticamente.

4. **Criar UI em SwiftUI**
   Lista, filtros, busca, bottom sheet de adição. Os use cases são chamados diretamente do ViewModel Swift.

🚀 **O valor real do KMP neste projeto**

Quando o app iOS for construído, ele vai usar exatamente a mesma lógica que o Android. Mesmas categorias, mesmas regras de validação, mesmo comportamento de upsert, mesma query de ordenação. Zero risco de divergência de comportamento entre plataformas — porque é literalmente o mesmo código compilado para cada uma.

Comando para gerar o framework iOS

```
./gradlew :shared:linkDebugFrameworkIosSimulatorArm64
# Gera: shared/build/bin/iosSimulatorArm64/debugFramework/shared.framework
```

11

TECNOLOGIA · EVOLUÇÃO

Stack e Próximos Passos

Stack completa

Tecnologia	Versão	Papel no projeto
Kotlin	2.0.0	Linguagem principal — Android e shared
KMP	2.0.0	Compartilha domínio entre Android e iOS
Jetpack Compose	BOM 2024.09	UI declarativa Android
Material Design 3	—	Design system com suporte a dark mode
Room	2.6.1	Banco SQLite com Flow reativo
Koin	3.5.6	Injeção de dependências sem reflexão
Coroutines + Flow	1.8.1	Assincronismo e reatividade
KSP	2.0.0-1.0.21	Geração de código do Room em compile-time

Próximos passos sugeridos

🔴 **App iOS (alta prioridade)**

Completar o objetivo original do projeto. O domínio já está pronto — é "só" a UI.

🔴 **Sincronização em tempo real**

Firebase Firestore ou Supabase para sincronizar a lista entre os dois celulares. A pessoa que está em casa vê em tempo real o que foi marcado no mercado.

🟡 **Notificações push**

Avisar quando a lista for atualizada — "Leite foi adicionado à lista."

🟡 **Testes unitários**

Use cases em `commonMain` são testáveis com `kotlin.test` sem Android. Testar a lógica de filtragem e agrupamento do UiState.

🟢 **Histórico e sugestões**

Guardar itens já comprados para sugerir rapidamente na próxima lista.

O que este projeto demonstra

✅ **Competências demonstradas**

KMP: estrutura multiplataforma com expect/actual, framework iOS configurado, Flow em commonMain.

Arquitetura: Clean Architecture completa, Repository pattern, Use Cases com responsabilidade única.

Android moderno: Compose, Material 3, Room com Flow, Koin, StateFlow + combine(), collectAsStateWithLifecycle.

UX: animações, estados vazios, confirmações antes de ações destrutivas, feedback visual imediato.